

Relativistic Raytracer: A GPU Benchmark Across Metrics, Integrators, and Platforms

Artur Raffael Cavalcanti

Universidade Federal Rural de Pernambuco (UFRPE)

Recife, Brazil

Email: artur.raffael@ufrpe.br

Lucas Silva Figueiredo

Universidade Federal Rural de Pernambuco (UFRPE)

Recife, Brazil

Email: lucas.sfigueiredo@ufrpe.br

Abstract—Interactive relativistic ray tracing has shifted from offline pipelines to real-time GPU workloads, allowing the visualization of gravitational light deflection via per-pixel geodesic integration. However, developers face critical engineering tradeoffs between high-level engine abstractions (e.g., Unity) and low-level APIs (e.g., Vulkan), as well as the choice of numerical integrators across different spacetime geometries. This paper introduces A Multi-Engine Benchmark for Relativistic Raytracing, a comprehensive performance-accuracy dataset spanning 16,656 timed renders across three spacetime metrics (Newton, Schwarzschild, Kerr), four integrators, and six scenes of varying multi-body complexity. Our findings reveal a resolution-dependent platform crossover: Unity outperforms Vulkan at ultra-low scales (144p) due to dispatch-overhead dominance, whereas Vulkan achieves up to a $2\times$ throughput advantage at interactive resolutions. Furthermore, we isolate the computational impact of numerical integration, identifying the first-order Euler method with a medium step size ($h = 52\text{ km}$) as a highly practical sweet spot ($>1000\text{ fps}$, $\text{SSIM} \approx 0.57$) for interactive Kerr rendering. Conversely, we demonstrate that monotonic error degradation under extreme mass ($> 10^{33}\text{ kg}$) and spin parameters renders the fourth-order Runge-Kutta (RK4) scheme mandatory for numerical accuracy. Additionally, we expose multi-body scene complexity as a benchmark dimension, proving that the Euler-to-RK4 performance gap scales arithmetically with the number of gravitational sources, growing from $105\times$ for a single body up to $191\times$ for a ten-body solar system. This benchmark serves as reference for balancing performance and image quality in relativistic computer graphics.

I. INTRODUCTION

In relativistic ray tracing, the linear ray-generation pipeline is extended to approximate General Relativity, bending the ray to simulate gravitational light deflection in spacetime. [1] Programmable GPUs have shifted the frontier, enabling per-pixel geodesic integration in real time of what once was confined to offline pipelines [2] and making relativistic visualisation accessible for interactive scientific, educational and entertainment tools [3], [4]. In this work, we focus on the real-time rendering of black holes. Practitioners implementing such systems face interconnected engineering decisions. **Platform**: a high-level engine (e.g., Unity) reduces development friction through abundant community resources and camera controls, but imposes scheduling runtime overhead; a low-level API (e.g., Vulkan) demands explicit resource management yet exposes full compute throughput. **Integrator**: to solve the geodesic ordinary differential equation, we contrast the first-order Euler method against the fourth-order Runge-Kutta

(RK4) scheme, evaluating how discretization orders impact performance. Beyond these core pillars, configurations like integrator step sizes, maximum step limits, gravitational multipliers, body spin parameters, and camera viewpoints heavily influence the final render.

Prior interactive renderers report isolated timing results [5], [6], making cross-platform or cross-integrator performance comparisons difficult to isolate. This work addresses this gap by executing a comprehensive, factorial performance-accuracy benchmark that evaluates these architectural decisions and scene parameters simultaneously across thousands of controlled renders (Figure 1).

Our contributions are:

- A **direct comparison of Unity and Vulkan** as platforms for GPU relativistic raytracing, across six scenes, three resolutions (144p, 480p, 720p), and 16,656 timed renders.
- An evaluation of **three spacetime metrics**: Newton heuristic, Schwarzschild, and Kerr (rotating black hole). Under four integrators (Euler at step sizes of 260, 52, and 1 km; and RK4 at 1 km step), Isolating per-metric cost relative to the integrator’s overhead.
- A **quantitative and visual quality benchmark** using Peak signal-to-noise ratio (PSNR) and Structural similarity index measure (SSIM) alongside final render outcomes across all configurations. This serves as a practical reference, helping developers visualize the image-quality impact of different step sizes and spacetime conditions.

II. FOUNDATIONS

A. Spacetime Metrics

Unlike Newtonian mechanics, General Relativity models gravity as the curvature of spacetime, forcing all moving particles, including light, to follow geodesics. These locally shortest paths are governed by the geodesic equation:

$$\frac{du^\mu}{d\lambda} = -\Gamma_{\nu\sigma}^\mu u^\nu u^\sigma, \quad (1)$$

where $u^\mu = dx^\mu/d\lambda$ is the four-velocity and $\Gamma_{\nu\sigma}^\mu$ are the Christoffel symbols. In a rendering pipeline, these symbols act as position-dependent transformation weights that dictate the trajectory of the ray, and their evaluation dominates the shader’s computational cost.

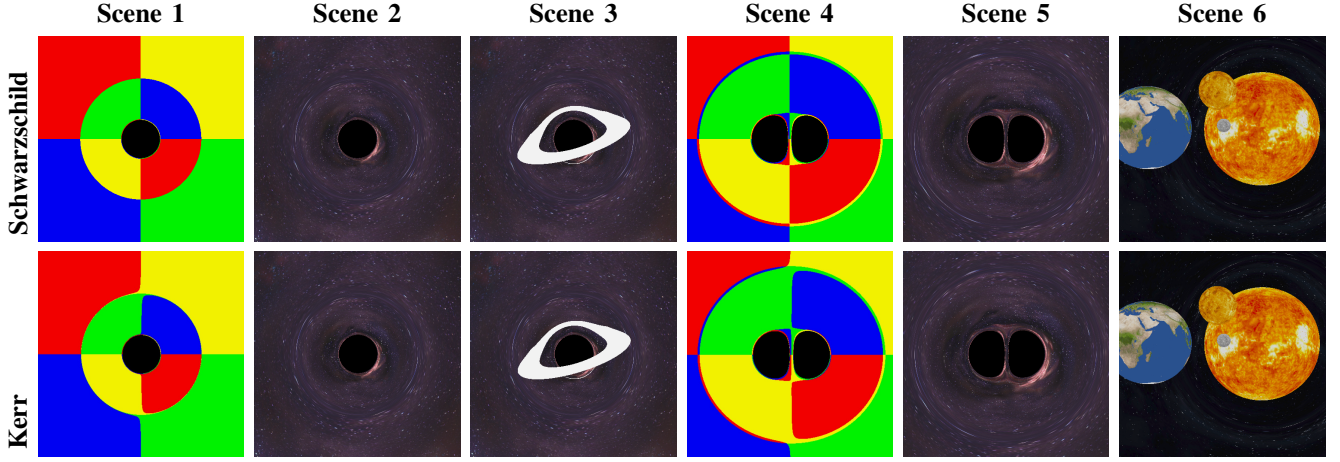


Fig. 1. Six benchmark scenes (columns) across two metrics (rows), using 480p resolution, Vulkan platform, Normal Gravity, and RK4 integrator. *Schwarzschild* (top): broad smooth lensing, no event horizon or photon sphere. *Kerr* (bottom): frame-dragging breaks photon-ring symmetry with asymmetric Doppler brightening. Scenes 1–3: single black hole; Scenes 4–5: binary (2 bodies); Scene 6: ten-body solar system (Moon, Earth, and Sun visible as textured spheres).

Newton (heuristic). Applies classical gravitational acceleration $\mathbf{a} = -(GM/r^2)\hat{\mathbf{r}}$ as a per-step deflection. It forces light to curve but serves only as a non-relativistic baseline.

Schwarzschild. Describes a static, spherically symmetric black hole [1]:

$$ds^2 = -\left(1 - \frac{r_s}{r}\right) c^2 dt^2 + \left(1 - \frac{r_s}{r}\right)^{-1} dr^2 + r^2 d\Omega^2, \quad (2)$$

where $r_s = 2GM/c^2$. This metric models accurate gravitational lensing (i.e., severe spatial distortion of background light) for non-rotating bodies by curvature of spacetime.

Kerr. Generalises Schwarzschild to a rotating mass with spin parameter $a = J/(Mc)$ [1]:

$$ds^2 = -\left(1 - \frac{r_s r}{\Sigma}\right) c^2 dt^2 - \frac{2r_s r a \sin^2 \theta}{\Sigma} c dt d\phi + \frac{\Sigma}{\Delta} dr^2 + \Sigma d\theta^2 + \left(r^2 + a^2 + \frac{r_s r a^2 \sin^2 \theta}{\Sigma}\right) \sin^2 \theta d\phi^2, \quad (3)$$

where $\Sigma = r^2 + a^2 \cos^2 \theta$ and $\Delta = r^2 - r_s r + a^2$. This geometry not only bends but twists spacetime via frame-dragging (an asymmetric swirling effect that forces light rays to spiral around the equator). Consequently, Kerr contains substantially more non-zero Christoffel components, making it the most computationally demanding metric in this benchmark. The qualitative visual differences between the symmetric Schwarzschild lensing and the asymmetric Kerr frame-dragging across our configurations are shown in Figure 1.

B. Numerical Integrators

Euler uses a single metric evaluation per step:

$$\mathbf{y}_{n+1} = \mathbf{y}_n + h f(\lambda_n, \mathbf{y}_n), \quad (4)$$

where $\mathbf{y} = (x^\mu, u^\mu)$ and h is the step size. Global error is $\mathcal{O}(h)$; large steps near the photon sphere produce deterministic dark-pixel voids. We test three step sizes: Large ($h = 260 \text{ km} \approx 17.6 M$), Medium ($h = 52 \text{ km} \approx 3.5 M$), Small

($h = 1 \text{ km} \approx 0.07 M$), where $M = 10 M_\odot$ ($M_\odot$ denotes solar mass) is the canonical scene mass.

RK4 evaluates four substeps per integration step [7]:

$$\begin{aligned} k_1 &= h f(\lambda_n, \mathbf{y}_n), \\ k_2 &= h f\left(\lambda_n + \frac{1}{2}h, \mathbf{y}_n + \frac{1}{2}k_1\right), \\ k_3 &= h f\left(\lambda_n + \frac{1}{2}h, \mathbf{y}_n + \frac{1}{2}k_2\right), \\ k_4 &= h f(\lambda_n + h, \mathbf{y}_n + k_3), \\ \mathbf{y}_{n+1} &= \mathbf{y}_n + \frac{1}{6}k_1 + \frac{1}{3}k_2 + \frac{1}{3}k_3 + \frac{1}{6}k_4 + \mathcal{O}(h^5). \end{aligned} \quad (5)$$

Global error is $\mathcal{O}(h^4)$, reducing artifacts at the cost of four Christoffel evaluations per step. We use $h = 1 \text{ km}$ for RK4.

III. RELATED WORK

Luminet [8] computed the canonical Schwarzschild black-hole image with a thin accretion disk; Riazuelo [9] revisited Schwarzschild ray tracing for physically faithful star-field and horizon rendering. Psaltis & Johannsen [10] provide a ray-tracing formulation underlying many later implementations. These foundational image-synthesis papers established the visual phenomena later systems reproduce. On the GPU scientific side, GRay [2] and GRay2 [11] demonstrated high-throughput Kerr geodesic integration; Odyssey [12] extended the direction to general-relativistic radiative transfer in Kerr spacetime, while Blacklight [13], Skylight [14], and OSIRIS [15] represent more recent tools for simulation post-processing, arbitrary spacetimes, and compact-object imaging.

A second branch emphasises graphics, visualization, and interactive use. Kuchelmeister et al. [5] implemented four-dimensional GPU ray tracing with timing results for Schwarzschild and Kerr scenes; Müller & Frauendiener [6] demonstrated interactive Schwarzschild exploration on desktop hardware; GeoViS [16] broadened support to multiple analytic spacetimes. Verbraeck & Eisemann [3] and Bruneton [17] pursued faster Schwarzschild via precomputation or approximation; Astray [18] treats implementation portability as a systems goal. At the application end, Yamashita [4] presented

a lightweight black-hole game and GravLensX [19] explored a learned surrogate for null-geodesic tracing.

Validation in relativistic rendering relies on analytical expectations or higher-precision reference codes. Constants of motion and the Kerr photon region provide qualitative sanity checks [1]; Müller & Weiskopf [20] discuss their visual signatures in shadow size and frame-dragging asymmetry. Vincent et al. [21] introduced GYOTO for open general-relativistic ray tracing; Bohn et al. [22] apply the same logic to binary-black-hole merger rendering. Regarding application scenarios, examples include: *Interstellar*’s DNGR pipeline [23] demonstrated Kerr lensing for film, and Weiskopf et al. [24] and Kersting [25] show its value for education. Novello et al. [26]–[28] study curved-space rendering in Riemannian settings, methodologically adjacent but not direct prior art for black-hole geodesic benchmarking. Furthermore, visualizing complex Thurston geometries lacking closed-form geodesic solutions, such as Sol and $SL_2(R)$, heavily relies on similar numerical integration techniques, demonstrating the broader utility of Euler methods in non-Euclidean graphics [29].

IV. BENCHMARK DESIGN

A. Platforms

Unity (6000.2.14f1, IL2CPP) replaces the rasterization pipeline with a HLSL `FragmentShader` that launches one geodesic integration per pixel. `RayTracingManager` transfers camera matrices and gravitational parameters to the GPU via shader uniforms; `BenchmarkConfiguration` organises presets for deterministic execution. Frame time is recorded as the total end-to-end latency per frame under an uncapped frame rate. This approach encapsulates the full processing pipeline, accounting for engine runtime overhead, driver command-buffer scheduling, and GPU integration.

Vulkan (SDK 1.3.275, C++17) implements a compute pipeline to manage execution, dispatching work via `vkCmdDispatch` to a 16×16 -thread workgroup grid. Scene parameters are transmitted through a `CameraUBO`, and the output is written directly to a storage image before being copied to the swapchain. Frame timing is evaluated under a four-frame in-flight architecture using CPU wall-clock ticks, capturing the end-to-end latency similarly to the high-level engine configuration to ensure a consistent comparison.

All simulations used a single workstation (NVIDIA GeForce RTX 3050, AMD Ryzen 5 3500X @ 3.6 GHz, 24 GB DDR4, Windows 11 Pro) with VSync disabled and target frame rate uncapped. Each configuration ran for 4 s (1 s warm-up, 3 s recording).

B. Scenes

Table I lists six scenes with increasing gravitational complexity. Scenes 1–3 use a single black hole; Scenes 4–5 use binary black holes; Scene 6 is a ten-body solar-system stress test with a Milky Way skybox.

Scenes 1–3 evaluate a single black hole with, respectively, a colored background, an astrophysical skybox, and the same skybox with an accretion disk. Scenes 4 and 5 transition to

TABLE I
BENCHMARK SCENES.

Scene	Description
1	Single black hole, colored background
2	Single black hole, universe skybox
3	Single black hole, skybox + accretion disk
4	Binary black holes, colored background
5	Binary black holes, universe skybox
6	Ten-body solar system, Milky Way skybox

TABLE II
FACTORIAL DESIGN FACTORS AND LEVELS.

Factor	Levels
Platform	Unity, Vulkan
Metric	Newton, Schwarzschild, Kerr
Integrator	Euler-L/M/S ($h=260/52/1$ km), RK4 ($h=1$ km)
Resolution	144p, 480p, 720p
Scene	1–6
Gravity	Normal, Strong, Max
Spin (Kerr)	Moderate, High, Very High, Extreme [†]

[†] Extreme spin ($a=200 M$) is a numerical stress test; pixel-identical all-black pairs excluded from PSNR computation.

a binary system, testing it against a colored background and a skybox. Finally, Scene 6 serves as a ten-body solar system stress test, significantly increasing the number of bodies to induce a high multi-body computational load.

Physical note: Scenes 4–6 superimpose independent Schwarzschild or Kerr metrics, which is not a solution to the Einstein field equations for those configurations.

C. Factorial Design

The benchmark crosses the factors in Table II, producing **16,656** timed renders each paired with a PNG image. Image quality (PSNR and SSIM) is evaluated comparing the 10,656 Euler integrators paired with RK4 as the high-accuracy reference.

V. RESULTS

The comprehensive benchmark dataset is publicly available and hosted on the Zenodo repository (zenodo.org/records/20562083). Our Unity and Vulkan source codes are available at github.com/tucoff/relativistic-raytracer-unity and github.com/tucoff/relativistic-raytracer-vulkan, respectively.

A. Platform Performance

At 144p, Unity outperforms Vulkan by $1.4\text{--}1.6\times$ (fps ratio). At sub-millisecond GPU kernel durations, the measured frame time is dominated by CPU-side loop overhead and runtime environment scheduling rather than shader execution. At this ultra-low resolution scale, the comprehensive engine architecture of Unity manages command synchronization and window system events with highly optimized framing intervals that minimize driver idling. The relationship inverts at 480p and above, where Vulkan leads by $1.1\text{--}1.3\times$ for single-body scenes

TABLE III

RENDER TIME (MS) AT 480P, VULKAN, SCENE 1 (FRONTAL CAMERA). E-L/M/S: EULER STEP SIZES 260/52/1 KM; RK4 AT $h=1$ KM. SPEED-UP ($\times$): EULER-L / RK4 RATIO. INTERACTIVE THRESHOLD: 33.3 MS (30 FPS).

Metric	E-L	E-M	E-S	RK4	$\times$
Newton	0.39	0.73	18.0	37.2	95 $\times$
Schwarzschild	0.39	0.80	20.2	39.8	102 $\times$
Kerr	0.38	0.90	21.3	49.4	130 $\times$

E-L and E-M are interactive at all tested resolutions and metrics. E-S is interactive at 144p and 480p. RK4 is interactive only at 144p.

TABLE IV

IMAGE QUALITY VS. RK4 AND THROUGHPUT SPEED-UP (VULKAN, 480P, MEAN OVER ALL SCENE/GRAVITY/SPIN CONFIGURATIONS). PSNR EXCLUDES PIXEL-IDENTICAL PAIRS. σ REFLECTS GENUINE SCENE AND SPIN HETEROGENEITY. BOLD: WORST OVERALL CONFIGURATION.

Metric	Integrator	PSNR (dB)	SSIM ($\mu \pm \sigma$)	Speed-up
Newton	Euler-L	15.4	0.60 $\pm$ 0.26	105 $\times$
	Euler-M	18.5	0.66 $\pm$ 0.26	63 $\times$
	Euler-S	34.9	0.88 $\pm$ 0.23	2.8 $\times$
Schw.	Euler-L	13.2	0.54 $\pm$ 0.24	112 $\times$
	Euler-M	16.5	0.66 $\pm$ 0.22	49 $\times$
	Euler-S	31.4	0.91 $\pm$ 0.12	2.3 $\times$
Kerr	Euler-L	12.4	0.47$\pm$0.25	141 $\times$
	Euler-M	15.1	0.57 $\pm$ 0.26	66 $\times$
	Euler-S	25.8	0.78 $\pm$ 0.29	2.5 $\times$

and up to 2.0 $\times$ in the ten-body Scene 6, where heavy multi-body Christoffel summation raises GPU frame time into the tens-of-milliseconds regime.

B. Integrator Accuracy Tradeoff

Throughput. Euler-Large delivers sub-millisecond frame times (< 0.5 ms, > 2000 fps at 480p) for all three metrics (Table III). RK4 approaches interactive rates only at 480p (≈ 20 – 27 fps); at 720p it is non-interactive (> 71 ms). Euler-Medium occupies a useful middle tier: > 1000 fps at 480p for all metrics. The Euler-L to RK4 throughput gap spans 95 $\times$ (Newton) to 130 $\times$ (Kerr); it widens with metric complexity because RK4’s fourfold Christoffel cost amplifies the Kerr overhead while Euler-L is bandwidth-saturated and effectively metric-agnostic.

Quality. SSIM degrades with step size: 0.88–0.91 (Euler-S), 0.54–0.66 (Euler-M), 0.47–0.60 (Euler-L). Kerr/Euler-Large is the worst configuration (PSNR 12.4 dB, SSIM 0.47); frame-dragging tightens the photon region so the 260 km step misses near-circular geodesics deterministically (Table IV). Figure 2 documents the visual progression at extreme spin parameter.

C. Scene Complexity and Multi-Body Scaling

The Euler-L/RK4 fps ratio grows with body count (Table V): 104–106 $\times$ (single body), 163 $\times$ (binary), 191 $\times$ (ten-body solar system). Each Christoffel evaluation in a multi-

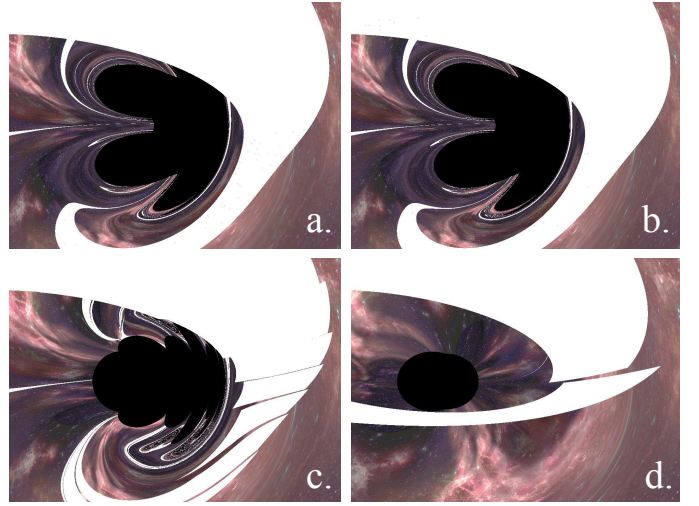


Fig. 2. Integrator vs. quality. (a) RK4: continuous photon ring. (b) Euler-S: visually indistinguishable from RK4. (c) Euler-M: slicing pattern of the photon ring. (d) Euler-L: single slice and loss of the distortion dimension.

TABLE V

PER-SCENE FPS AT 480P (VULKAN, KERR, CANONICAL GRAVITY AND SPIN). RATIO = EULER-L / RK4. SCENES 1–3: SINGLE BODY; 4–5: BINARY (2 BODIES); 6: TEN-BODY SOLAR SYSTEM.

Sc.	Configuration	RK4	Euler-L	Ratio
1	Single BH, colored bg.	245	2619	106 $\times$
2	Single BH, skybox	245	2554	104 $\times$
3	Single BH, rings	242	2566	105 $\times$
4	Binary BH, colored bg.	35	2587	163 $\times$
5	Binary BH, skybox	35	2577	163 $\times$
6	Solar system (10 bodies)	6	867	191 $\times$

body scene must sum gravitational contributions from all N_{bodies} sources, so per-step cost scales as $4 \times N_{\text{bodies}}$ for RK4 and N_{bodies} for Euler. On the other hand, Figure 3 shows how Euler-L quality (revealed by the PSNR) degrades with gravitational mass increase, dropping from 24.5 dB to 7.0 dB (from 10^{31} to 10^{34}). Interestingly, Figure 4 shows that for these binary black holes, the PSNR increases from 12.0 dB to 15.1 dB as spin increases from natural to extreme.

D. Stress-Test Qualitative Evaluation

Figure 5 illustrates extreme general-relativistic phenomena across diverse configurations, clearly manifesting the distinct physical signatures of frame-dragging and gravitational lensing, while also manifesting resolution-limiting noise and severe ray-divergence (aliasing artifacts) typical of highly non-linear optical regimes.

VI. DISCUSSION

The 144p crossover. At 144×256 pixels, GPU kernels complete in well under 1 ms for all integrators; measured frame time is dominated by CPU-side loop overhead and runtime scheduling rather than shader execution. At this

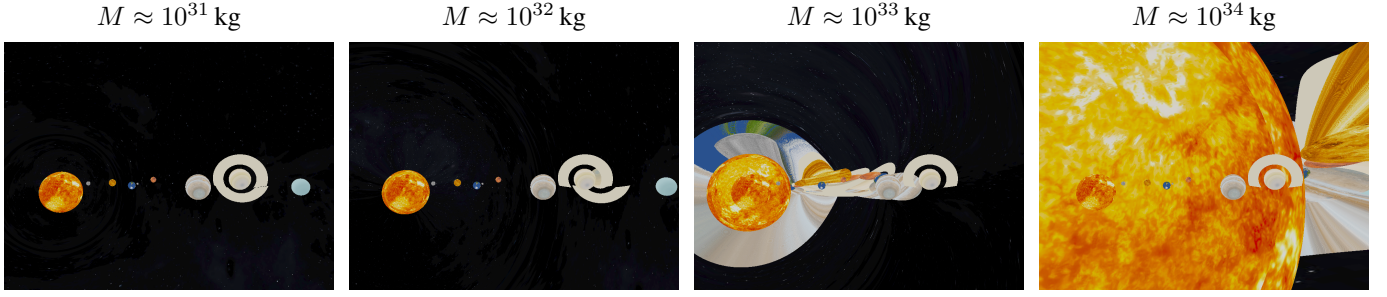


Fig. 3. Euler-Large image quality vs. gravitational mass (Newton, Scene 6, moderate spin, 720p, Vulkan). PSNR: 24.5 $\rightarrow$ 13.4 $\rightarrow$ 6.4 $\rightarrow$ 7.0 dB as mass grows from 10^{31} to 10^{34} kg.

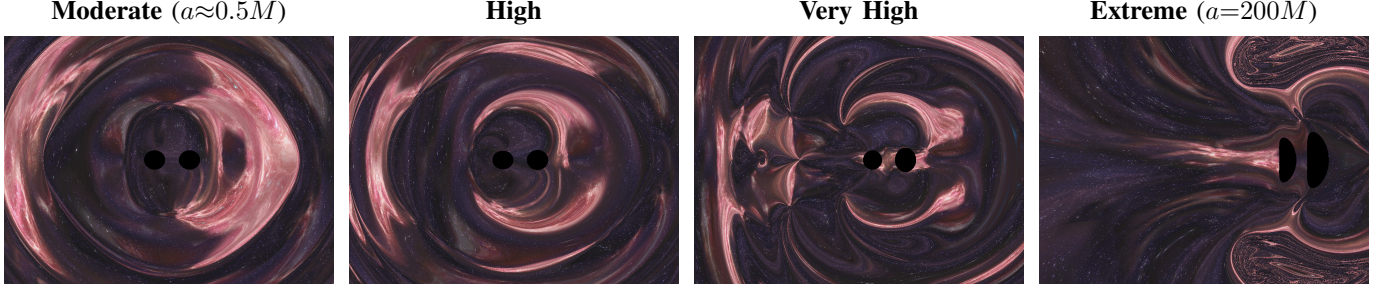


Fig. 4. Euler-Large image quality vs. spin parameter (Kerr, Scene 5, binary black holes, Strong G, 720P, Vulkan). PSNR: 12.0 $\rightarrow$ 13.3 $\rightarrow$ 13.4 $\rightarrow$ 15.1 dB from moderate to extreme spin. Increasing frame-dragging wraps background light into spirals and contracts the photon ring asymmetrically.

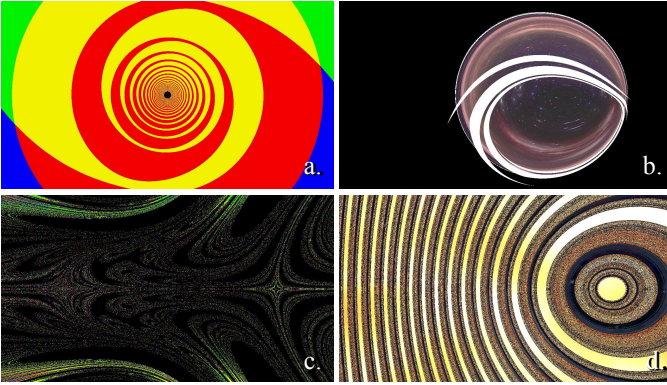


Fig. 5. Stress-test gallery (Vulkan). (a) Scene 1, Euler-S, Normal G, extreme spin ($a=200$), above-camera: frame-dragging spirals the background into dense concentric rings. (b) Scene 3, RK4, extreme gravity, look-away camera: strong curvature deflects outward-propagating light, making the accretion disk visible from behind the black hole. (c) Scene 4, Euler-S, extreme spin, look-away: binary frame-dragging sweeps quadrant boundaries into interleaved geodesic vortices. (d) Scene 6, RK4, $M=10^{34}$ kg, extreme spin: extreme mass compresses the skybox into dense rings; solar-system bodies survive as bright ovals in the lensing halo.

ultra-low scale, Unity’s highly optimized engine framework handles windowing and command synchronization interfaces with tight integration intervals that minimize driver idling. Above 144p, GPU execution time dominates the pipeline, and Vulkan’s thinner driver abstraction yields a persistent advantage, up to $2.0\times$ in the computationally dense ten-body Scene 6, where heavy multi-body Christoffel summation raises execution times. This crossover was not reported by prior interactive renderers [5], [6], which operated exclusively at

resolutions where GPU execution dominates; it remains a practical design concern only for very-low-resolution contexts such as thumbnail generation or editor viewports.

Euler-Medium sweet spot. For interactive Kerr rendering, Euler-Medium ($h = 52$ km) achieves $\approx 66\times$ the throughput of RK4 at 480p while yielding SSIM 0.57 and PSNR 15.1 dB, a gain of $+0.10$ SSIM over Euler-Large at $2.4\times$ the render cost (0.38 ms to 0.90 ms per frame). For Newton and Schwarzschild, Euler-Medium produces no visible photon-ring fragmentation; the deterministic ring holes are a Kerr-specific concern arising from the tighter photon region induced by frame-dragging. The reported means carry substantial spread ($\sigma_{\text{SSIM}} \approx 0.25\text{--}0.26$; Table IV), so these operating points should be treated as fleet averages over all scene and spin configurations.

Strong-field RK4 threshold. For Scenes 1–5 at canonical gravity levels, Euler-Large Kerr achieves PSNR ≈ 12.7 dB and SSIM ≈ 0.51 , a stable, moderate quality suitable for non-scientific interactive use. Scene 6’s mass sweep reveals a severe quality degradation: PSNR falls from 24.5 dB (10^{31} kg) to 6.4 dB (10^{33} kg), stabilizing around 7.0 dB (10^{34} kg; Figure 3). Visual legibility collapses below ≈ 7.5 dB, placing the RK4-mandatory threshold at $M \approx 10^{33}$ kg. Interestingly, high spin does not compound this degradation under large step sizes; instead, Figure 4 shows the Euler-L PSNR increasing from 12.0 dB (natural spin) to 15.1 dB (extreme spin).

Multi-body dimension. Our benchmark reports fps under controlled multi-body geodesic integration. Our results show that the Euler/RK4 speed advantage is largest precisely when the simulation is most physically complex (Euler-L/RK4 ratio:

105× single body, 191× ten bodies). The mechanism is arithmetic: RK4 pays $4 \times N_{\text{bodies}}$ Christoffel evaluations per step; Euler pays only N_{bodies} . This scaling behaviour motivates adaptive integrators [7] and learned geodesic surrogates [19] for future multi-body interactive systems.

Limitations. (1) Simulations use a single GPU; fps ratios between Vulkan and Unity are expected to vary with GPU generation and driver maturity. (2) Fixed step sizes are used throughout; adaptive control (e.g., Dormand-Prince RK4/5 [7]) would reduce cost in low-curvature regions and could close the Euler–RK4 quality gap at lower compute. (3) Multi-body scenes apply superimposed independent metrics, not exact multi-body GR solutions; results quantify the scaling of the additive approximation.

VII. CONCLUSION

This paper established a robust benchmark for relativistic GPU ray tracing, demonstrating that low-level APIs like Vulkan outperform high-level engines at interactive resolutions by exposing full compute throughput, particularly in complex multi-body scenarios. Through extensive evaluations, we identified the first-order Euler method with a medium step size ($h = 52$ km) as a highly practical baseline. While minimizing the integration step size expectedly improves numerical convergence at the expense of higher computational overhead, our stress tests under extreme gravitational fields ($M > 10^{33}$ kg) and high spin parameters prove that higher-order schemes, such as the fourth-order Runge-Kutta (RK4) method, remain mandatory to preserve numerical accuracy.

Future work should explore optimization strategies like adaptive step-sizing for RK4 to reduce computational load while preserving precision. Ultimately, these insights establish a foundation for balancing performance and visual quality in interactive astrophysics, education, and entertainment. By open-sourcing our code and reference imagery, this work serves as an accessible entry point—bridging the gap between physicists learning programming and graphics programmers exploring relativity.

REFERENCES

- [1] S. M. Carroll, *Spacetime and Geometry: An Introduction to General Relativity*. San Francisco: Addison-Wesley, 2004.
- [2] C.-k. Chan, D. Psaltis, and F. Özel, “GRay: A massively parallel GPU-based code for ray tracing in relativistic spacetimes,” *The Astrophysical Journal*, vol. 777, no. 1, p. 13, 2013.
- [3] A. Verbraeck and E. Eisemann, “Interactive black-hole visualization,” *IEEE Transactions on Visualization and Computer Graphics*, vol. 27, no. 2, pp. 796–805, 2021.
- [4] Y. Yamashita, “Development of a chase game in a black hole spacetime,” *Jxiv Preprints*, 2026.
- [5] D. Kuchelmeister, T. Müller, M. Ament, G. Wunner, and D. Weiskopf, “GPU-based four-dimensional general-relativistic ray tracing,” *Computer Physics Communications*, vol. 183, no. 10, pp. 2282–2290, 2012.
- [6] T. Müller and J. Frauendiener, “Studying null and time-like geodesics in the classroom,” *European Journal of Physics*, vol. 32, no. 3, pp. 747–759, 2011.
- [7] W. H. Press, S. A. Teukolsky, W. T. Vetterling, and B. P. Flannery, *Numerical Recipes: The Art of Scientific Computing*, 3rd ed. Cambridge: Cambridge University Press, 2007.
- [8] J.-P. Luminet, “Image of a spherical black hole with thin accretion disk,” *Astronomy and Astrophysics*, vol. 75, pp. 228–235, 1979. [Online]. Available: <https://ui.adsabs.harvard.edu/abs/1979A&A....75..228L>

- [9] A. Riazuelo, “Seeing relativity – i. ray tracing in a Schwarzschild metric to explore the maximal analytic extension of the metric and making a proper rendering of the stars,” *International Journal of Modern Physics D*, vol. 28, no. 2, p. 1950042, 2019. [Online]. Available: <https://arxiv.org/abs/1511.06025>
- [10] D. Psaltis and T. Johannsen, “A ray-tracing algorithm for spinning compact object spacetimes with arbitrary quadrupole moments. I. quasi-Kerr black holes,” *The Astrophysical Journal*, vol. 745, no. 1, p. 1, 2012.
- [11] C.-k. Chan, L. Medeiros, F. Özel, and D. Psaltis, “GRay2: A general purpose geodesic integrator for Kerr spacetimes,” *The Astrophysical Journal*, vol. 867, no. 1, p. 59, 2018.
- [12] H.-Y. Pu, K. Yun, Z. Younsi, and S.-J. Yoon, “Odyssey: A public GPU-based code for general-relativistic radiative transfer in Kerr spacetime,” *The Astrophysical Journal*, vol. 820, no. 2, p. 105, 2016. [Online]. Available: <https://arxiv.org/abs/1601.02063>
- [13] C. J. White, “Blacklight: A general-relativistic ray-tracing and analysis tool,” *The Astrophysical Journal Supplement Series*, vol. 262, no. 1, p. 28, 2022. [Online]. Available: <https://arxiv.org/abs/2203.15963>
- [14] J. Pelle, O. Reula, F. Carrasco, and C. Bederian, “Skylight: a new code for general-relativistic ray-tracing and radiative transfer in arbitrary space-times,” *Monthly Notices of the Royal Astronomical Society*, vol. 515, no. 1, pp. 1316–1327, 2022. [Online]. Available: <https://arxiv.org/abs/2206.06429>
- [15] J. M. Velázquez-Cadavid, J. A. Arrieta-Villamizar, F. D. Lora-Clavijo, O. M. Pimentel, and J. E. Osorio-Vargas, “OSIRIS: a new code for ray tracing around compact objects,” *The European Physical Journal C*, vol. 82, no. 2, p. 103, 2022.
- [16] T. Müller, “GeoViS — relativistic ray tracing in four-dimensional spacetimes,” *Computer Physics Communications*, vol. 185, no. 8, pp. 2301–2308, 2014.
- [17] E. Bruneton, “Real-time high-quality rendering of non-rotating black holes,” arXiv preprint arXiv:2010.08735, 2020. [Online]. Available: <https://arxiv.org/abs/2010.08735>
- [18] C. Demiralp, M. Krüger, C. Chao, T. W. Kuhlen, and T. Gerrits, “Astray: A performance-portable geodesic ray tracer,” in *Proceedings of Vision, Modeling, and Visualization (VMV)*, 2022. [Online]. Available: <https://publications.rwth-aachen.de/record/956029>
- [19] M. Sun, Z. Fang, J. Wang, K. Zhang, Q. Zhang, and R. Xu, “Learning null geodesics for gravitational lensing rendering in general relativity,” in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2025. [Online]. Available: <https://arxiv.org/abs/2507.15775>
- [20] T. Müller and D. Weiskopf, “General-relativistic visualization,” *Computing in Science & Engineering*, vol. 13, no. 6, pp. 64–71, 2011.
- [21] F. H. Vincent, T. Paumard, E.ourgoulhon, and G. Perrin, “GYOTO: A new general relativistic ray-tracing code,” *Classical and Quantum Gravity*, vol. 28, no. 22, p. 225011, 2011.
- [22] A. Bohn, W. Thrope, F. Hébert, K. Henriksson, D. Bunandar *et al.*, “What does a binary black hole merger look like?” *Classical and Quantum Gravity*, vol. 32, no. 6, p. 065002, 2015.
- [23] O. James, E. von Tunzelmann, P. Franklin, and K. S. Thorne, “Gravitational lensing by spinning black holes in astrophysics, and in the movie Interstellar,” *Classical and Quantum Gravity*, vol. 32, no. 6, p. 065001, 2015.
- [24] D. Weiskopf, M. Borchers, T. Ertl, M. Falk, O. Fechtig, R. Frank, F. Grave, A. King, U. Kraus, T. Müller *et al.*, “Explanatory and illustrative visualization of special and general relativity,” *IEEE Transactions on Visualization and Computer Graphics*, vol. 12, no. 4, pp. 522–534, 2006.
- [25] M. Kersting, “Visualizing four dimensions in special and general relativity,” in *Handbook of the Mathematics of the Arts and Sciences*. Springer, 2021, pp. 2003–2038.
- [26] L. Velho, V. da Silva, and T. Novello, “Immersive visualization of the classical non-euclidean spaces using real-time ray tracing in VR,” in *Graphics Interface 2020*, 2020. [Online]. Available: <https://openreview.net/forum?id=liBL1-afIJ>
- [27] T. Novello, V. da Silva, and L. Velho, “Global illumination of non-Euclidean spaces,” *Computers & Graphics*, vol. 93, pp. 61–70, 2020.
- [28] —, *GPU Ray Tracing in Non-Euclidean Spaces*, ser. Synthesis Lectures on Visual Computing. Springer, 2022.
- [29] T. Novello, V. da Silva, and L. Velho, “Visualization of nil, sol, and sl2(r) geometries,” *Computers Graphics*, vol. 91, pp. 219–231, 2020. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0097849320301187>